EMAIL SPAM DETECTOR: CHROME EXTENSION

WITH PYTHON BACKEND

Anchal Anchal 300364004 – Team Lead

Course

Course Name: 4495 – Applied Research Project

Section: 002

1. Introduction

This report summarizes the modifications and enhancements made to the Spam Detection Chrome Extension project. The goal was to modernize the backend model, upgrade the frontend technology stack, and enable Gmail integration.

2. System Overview

The Spam Detection Chrome Extension with Real-Time Email Scanning is designed to enhance email security by automatically analyzing incoming emails in Gmail to detect spam messages. The system consists of two main components:

Chrome Extension:

Uses **content.js** to monitor Gmail for new emails and extract subject lines and snippets.

Communicates with **background.js**, which sends email data to a backend API for analysis.

Highlights potential spam emails in red within the Gmail inbox.

Description of Work Done

1. MySQL Integration for Structured Storage

- Designed a relational database schema to store email metadata, spam classification results, and timestamps.
- Created MySQL tables.
- Wrote Python scripts to establish a secure connection with MySQL using mysql-connector-python.
- Ensured smooth insertion and retrieval of classification records during API processing.
- Enabled historical spam analysis and query-based retrieval from the SQL database.

2. Enhanced Chrome Extension for Email Extraction

- Successfully implemented email extraction directly from the Gmail interface using a refined content script in the Chrome Extension.

- Used DOM parsing to dynamically retrieve email sender, subject, and body.
- Created a manual "Scan Email" button for user-initiated classification.
- Bypassed security limitations by adhering to Chrome's manifest v3 and using permissions properly (e.g., activeTab, scripting, host_permissions).
- Explored OAuth authentication but retained manual parsing due to time constraints.

3. Spam Classification with XGBoost

- Finalized XGBoost model pipeline using TF-IDF vectorization and GridSearchCV for hyperparameter tuning.
- Achieved higher accuracy and faster inference times compared to previous models.
- Integrated XGBoost into the backend API, replacing older Naïve Bayes implementation.
- Logged classification outcomes with confidence scores and timestamps into MySQL.

- 4. Performance Improvements & Debugging
- Optimized Chrome Extension to handle large emails efficiently with lazy loading and snippet truncation.
- Reduced API latency by caching recent classifications using local storage.
- Refined frontend AngularJS components to improve user interaction with classification results.
- Performed multiple rounds of bug fixes related to CORS, token parsing, and Angular digest cycle issues.

7 Current Challenges and Next Steps

Challenges Faced:

- OAuth 2.0 integration with Gmail remains challenging due to token refresh and scope restrictions.
- Balancing real-time processing speed with model accuracy continues to be a trade-off.
- Cross-browser compatibility of the extension (tested only on Chrome so far).

Next Steps:

- Finalize OAuth 2.0 implementation for secure, automated Gmail integration.
- Improve backend with async handling for email queue processing.
- Add visual dashboard for historical spam analysis from MySQL records.
- Conduct final round of UI/UX testing and package the extension for the Chrome Web Store.

Repo Check-In of Implementation:

Backend: Added MySQL integration, finalized XGBoost pipeline, improved API logging.

Frontend: Refined AngularJS components implemented email extraction via Chrome content scripts.

Database: Created structured schema in MySQL and added insertion logic for spam results.

Documentation: Updated README with MySQL schema, Chrome extension flow, and XGBoost pipeline.

Work Date/Hours Logs

Date	Hours	Descriptions
Jan 14 2025	2.4	Started searching for the project topic
Jan 15 2025	2	Looking for suitable topic
Jan 17 2025	3	Collected Information and made document about the fundamentals
Jan 20 2025	4	Got some ideas from professor and started looking forward on project
Jan 22 2025	4	Search for the tools and language to use
Jan 23 2025	3.5	Started working on the project, created json files for plugin
Jan 25 2025	3.5	created html and css pages for the chrome extension and some changes in chrome plugin
Jan 27 2025	4	studied some topics from machine laeaning and tried to implement in the project
Jan 29 2025	3	Faced diccifulty to push files to github and imported libraries
Jan 31 2025	3.5	Learned Machine laeaning Bay's theorem and did some practice to implement in the project

Feb 02 2025	4	Learned how to create chrome extension and tried to create it
Feb 03 2025	3	Created chrome extension files and uploaded to chrome extension
Feb 05 2025	2.5	Found out errors encountered while data cleaning
Feb 6 2025	3	Pushed files to github
Feb 9 2025	1	Created Report and checked the files done
Feb 12,2025	2	Installed libraries like blinker click and colorama
Feb 16,2025	2	imported metadata worked on api
Feb 17,2025	3	Set ted Up the Backend (Flask API)
Feb 23,2025	3	Pushed files on github and made video to upload.
Feb 25,2025	4	Studied Chrome extension architecture and reviewed Gmail content script integration.
Feb 28,2025	3.5	Created Manifest.json, added permissions for Gmail, and set up basic Chrome extension structure.
March 2,2025	3	Started development of content.js for scanning Gmail inbox. Implemented email extraction from Gmail UI and tested script injection.
March 4,2025	4	Worked on background.js to handle email content processing and API communication.
March 6,2025	3.75	Developed Flask API to receive email data for spam classification.
March 7,2025	4	Saved trained model as spam_model.pkl and integrated with Flask API again.
March 9,2025	4.2	Debugged API integration, identified issues with joblib vs pickle loading. Implemented MutationObserver in content.js to detect live email updates.
March 10 ,2025	3	Conducted initial testing, discovered API response lag issues. Optimized API response time by refining text preprocessing steps.
March 12 2025	3	Implemented email highlighting feature for detected spam messages. Deployed Flask API temporarily using ngrok for public access testing.

March 13 ,2025	4	Debugged scikit-learn version conflicts, considered model retraining. Refined spam classification pipeline, improved accuracy metrics.
March 15,2025	3.75	Integrated all extension components and tested Gmail real-time scanning. Conducted debugging and performance tuning for API interactions.
March 16,2025	5	Pushed files to git and made report. Tried to resolve errors and run API.
Mar 19, 2025	3	Researched and compared alternative browser automation tools (e.g., Puppeteer vs. Playwright) for potential integration into the spam detection pipeline.
Mar 20, 2025	3	Designed a fallback mechanism for email extraction when DevTools-based methods fail. Began prototyping a simplified heuristic parser for HTML email content.
Mar 21, 2025	2.5	Started implementing rate-limiting logic for API requests to prevent browser lag and avoid spam filter triggers during testing.
Mar 22, 2025	3	Developed and tested a feature to tag spam confidence scores on the frontend with visual indicators (e.g., color-coded flags).
Mar 23, 2025	4	Refactored project structure for better maintainability. Modularized email extraction logic and restructured model loading pipeline to support future updates. Also made report 3.
Mar 24, 2025	4	Reviewed initial project structure, identified tech stack changes needed.
Mar 25, 2025	4.5	Replaced ML model with XGBoost, wrote training and saving logic.
Mar 26, 2025	3.5	Updated Flask backend (app.py) to use XGBoost and serve predictions.
Mar 27, 2025	2	Built AngularJS frontend files and tested integration with backend API.
Mar 28, 2025	5	Researched Gmail API, set up credentials, and began script for email fetching.
Mar 29, 2025	4.5	Tried and tested gmail_fetcher.py script.

Mar 30, 2025	4	Compiled report and documented worklog. Final testing and packaging.
Mar 31st, 2025	5.5	Compared performance of trained models using precision, recall, accuracy, and ROC curves.
1st April, 2025	3.5	Modified predict.py to load my trained spam detection models (Naive Bayes and XGBoost) and vectorizer using pickle. I also created a clean_text function to preprocess email content and a classify_mail function that predicts whether a message is spam (1) or not spam (0). I tested it with a sample spam message to verify both models work correctly.
2nd April, 2025	4.5	Created function to expose two API endpoints (/classify and /emails/process) to classify emails as spam and log processed emails. Which uses a machine learning model (classify_mail) to analyze email content and stores results in a MySQL database. Created backend.js to support Gmail and Chrome extension by allowing secure cross-origin requests (CORS) and logging activities for debugging.
3rd April, 2025	6	Created check.py to quickly load and inspect the Enron email dataset. I used it to verify the column names and check the first few rows to understand the data format before building the spam detection model. This helped me confirm the structure and clean the data properly.
4th April, 2025	5	Created sample email data for local testing of the extension's detection accuracy.
5th April, 2025	5.5	Linked popup UI to backend detection logic and displayed results to the user. Styled popup interface with clean and user-friendly design. Built helper functions for email cleaning, tokenization, and vectorization.
6th April, 2025	5	Configured the Chrome extension by defining its name, permissions, and background script. Enabled access to Gmail via OAuth and specifies required scopes like reading and modifying emails. allowed the extension to run scripts on Gmail and interact with Google APIs.

Conclusion

This stage of the project marks a major milestone: integration of a structured database (MySQL), robust email extraction via a Chrome Extension, and the successful implementation of the high-performance XGBoost classifier. The foundation is now set for end-to-end testing, Gmail API integration, and UI polishing to complete the system for real-world deployment.